PyTorch Performance Tooling: Profiler & Kineto — Complete Documentation

> ***Contribution Note:*** *This document constitutes new and improved open-source documentation for the PyTorch repository, targeting the* `pytorch/tutorials` *and* `pytorch/pytorch` *repositories. It covers the* `torch.profiler` *module and the Kineto library in full, filling critical gaps in the existing documentation and providing high-quality tutorials for developers at all levels.*

Table of Contents

1. Introduction to Performance Profiling with PyTorch
2. The PyTorch Profiler Architecture: Kineto and CUPTI
3. Getting Started: Your First Profiling Session
4. Visualizing Traces with Perfetto
5. Advanced Profiling APIs and Techniques
6. Tutorial: From Profiling to Optimization — A Case Study
7. Tutorial: Profiling Distributed Training with HTA
8. Tutorial: Profiling Custom C++ and CUDA Operators
9. Advanced Topic: Memory Profiling with Memory Snapshots
10. Advanced Topic: Profiling `torch.compile`
11. API Quick Reference
12. References

---

1. Introduction to Performance Profiling with PyTorch

In the lifecycle of a deep learning model, achieving high accuracy is often the primary focus. However, once a model is ready for production, its performance in terms of speed and resource consumption becomes equally critical. A model that is too slow or memory-intensive can be impractical and costly to deploy. This is where **performance profiling** becomes an indispensable practice.

Performance profiling is the process of analyzing a program's execution to understand its behavior and identify bottlenecks. It involves measuring various aspects of the program, such as the time taken by different functions, memory usage, and hardware utilization. The goal is to pinpoint inefficiencies and areas for optimization, ultimately leading to a faster, more efficient model.

In the context of deep learning, profiling helps answer questions like: which operations in my model are the most time-consuming? Is my GPU being fully utilized, or is it sitting idle waiting for data? How much memory is my model using, and are there any unexpected spikes? Is the data loading pipeline a bottleneck? By answering these questions, developers can make informed decisions to optimize their models, leading to significant improvements in training and inference speed.

Common Performance Bottlenecks in Deep Learning

Deep learning models often exhibit a set of common performance issues. Understanding these can help you focus your profiling efforts effectively.

Bottleneck	Description	Potential Solutions
Data Loading	The data loading and preprocessing pipeline is too slow, causing the GPU to wait for data. This is often referred to as being “input-bound”.	Increase the number of workers in the `DataLoader`, use pinned memory, or optimize data augmentation operations.
GPU Idle Time	The GPU is not being kept busy with computation, leading to underutilization of expensive hardware.	Overlap data transfers with computation, use larger batch sizes, or fuse small operations into larger ones.
Inefficient Operators	Certain operations in the model are inherently slow or are not using the most efficient implementation for the target hardware.	Replace slow operators with more efficient alternatives, use mixed-precision training, or leverage `torch.compile`.
Memory Bottlenecks	The model consumes too much memory, leading to out-of-memory errors or limiting the batch size that can be used.	Use gradient checkpointing, reduce model size, or use memory-efficient optimizers.
Communication Overhead	In distributed training, the time spent synchronizing gradients across devices is too high.	Use gradient compression, overlap communication with computation, or tune the NCCL configuration.

---

2. The PyTorch Profiler Architecture: Kineto and CUPTI

To effectively use the PyTorch Profiler, it is helpful to understand its underlying architecture and the components that work together to collect and process performance data. The profiler is not a single monolithic entity but rather a system of interconnected parts, each with a specific role.

At a high level, the profiler’s architecture can be broken down into three main layers:

1. **The User-Facing API (`torch.profiler`)**: This is the primary interface that developers interact with. It provides the context managers and functions for starting, stopping, and configuring profiling sessions.

2. **Kineto**: The core library responsible for trace collection. It orchestrates the gathering of performance data from different sources, including the CPU and hardware accelerators. [1]
3. **Hardware-Specific Backends**: The low-level interfaces that Kineto uses to communicate with specific hardware, such as NVIDIA's CUPTI for CUDA GPUs.

The `torch.profiler` API

The `torch.profiler` module is the entry point for all profiling activities in PyTorch. It is designed to be intuitive and flexible, allowing you to profile your code with minimal changes. The most important component of this API is the `torch.profiler.profile` context manager, which you use to wrap the code you want to analyze.

When you enter the `profile` context, the profiler is activated. It then uses Kineto to start collecting data. The `activities` argument tells the profiler which types of events to record (e.g., CPU, CUDA). You can also use `record_function` to add custom labels to your code, making the profiling output easier to interpret.

Kineto: The Trace Collection Engine

Kineto is a C++ library that is responsible for the heavy lifting of trace collection. [1] When the profiler is active, Kineto receives notifications about events occurring in the PyTorch runtime, such as operator dispatches and kernel launches. It then records these events, along with their timestamps and other metadata, into a trace.

Kineto is designed to be extensible and can be integrated with various hardware backends. This is what allows the PyTorch Profiler to support a wide range of devices, not just NVIDIA GPUs.

Hardware-Specific Backends: CUPTI and Beyond

To collect data from a specific hardware accelerator, Kineto relies on a backend that can communicate with that hardware's performance monitoring tools. For NVIDIA GPUs, this backend is the **CUDA Profiling Tools Interface (CUPTI)**. CUPTI is a library provided by NVIDIA that allows developers to instrument and profile CUDA applications. Kineto uses CUPTI to subscribe to events occurring on the GPU, such as kernel launches and memory copies.

This modular architecture keeps PyTorch device-agnostic: Python brokers the session, the profiler translates profiler requests into backend runtime calls, and the runtime interacts with the accelerator. [2]

The Data Flow

The following table summarizes the data flow during a profiling session:

Step	Component	Action
1	User Code	Enters the `torch.profiler.profile` context manager
2	`torch.profiler`	Activates Kineto and configures the profiling session
3	PyTorch Runtime	Dispatches operators; Kineto records CPU-side events
4	CUDA Runtime	Launches GPU kernels
5	CUPTI	Detects kernel launches and notifies Kineto
6	Kineto	Records GPU-side events in the trace
7	`torch.profiler`	Finalizes the trace when the context is exited
8	User Code	Uses the `prof` object to analyze or export data

---

3. Getting Started: Your First Profiling Session

Now that we have a conceptual understanding of the PyTorch Profiler, let's walk through a practical example of how to use it. This section will guide you through the process of profiling a simple model, from setting up the code to interpreting the results.

Setting Up the Model and Data

First, let's create a simple model and some dummy data. We will use a ResNet18 model from `torchvision` and a random tensor as input.

```python
import torch
import torchvision.models as models

device = "cuda" if torch.cuda.is_available() else "cpu"
model = models.resnet18().to(device)
inputs = torch.randn(5, 3, 224, 224).to(device)
```

Using the `torch.profiler.profile` Context Manager

The easiest way to use the profiler is with the `torch.profiler.profile` context manager. For this example, we will profile both CPU and GPU activities and record the shapes of the operator inputs.

```python
import torch.profiler

with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
    ],
    record_shapes=True
) as prof:
    with torch.profiler.record_function("model_inference"):
        model(inputs)
```

The `record_function` context manager adds a custom label to the code block, which will appear in the profiling output. This is particularly useful for identifying specific parts of your model in the trace.

Analyzing the Results with `key_averages()`

Once the `profile` context is exited, the `prof` object contains all the collected performance data. The `key_averages()` method aggregates the results by operator name and provides a summary table.

```python
print(prof.key_averages().table(sort_by="cpu_time_total", row_limit=10))
```

Here is how to interpret the key columns in the output:

Column	Description
Name	The name of the operator or `record_function` label
Self CPU	Time in the operator itself, excluding time in child operators
CPU total	Total time in the operator, including time in child operators
CPU time avg	Average time per call
# of Calls	Number of times the operator was called
Self CUDA	Time on the GPU in the operator itself (if CUDA profiling is enabled)
CUDA total	Total time on the GPU, including child operators

Exporting a Trace for Visualization

While the `key_averages()` table is useful for a quick overview, a timeline visualization provides a much more detailed picture. The profiler can export the collected data into a Chrome-compatible trace file:

```
prof.export_chrome_trace("trace.json")
```

---

4. Visualizing Traces with Perfetto

The recommended tool for visualizing PyTorch Profiler traces is **Perfetto**, an open-source trace analysis tool developed by Google. [3] The TensorBoard plugin for PyTorch Profiler has been deprecated, and Perfetto is its replacement for all trace visualization needs.

> ***Note:*** *The TensorBoard integration with the PyTorch profiler (`torch.profiler.tensorboard_trace_handler`) is now deprecated. Instead, use Perfetto or the Chrome trace to view `trace.json` files. After generating a trace,*

> *drag the `trace.json` into [Perfetto UI](#) or `chrome://tracing` to visualize your profile.*[4]

Loading a Trace into Perfetto

To load your trace, navigate to ui.perfetto.dev in your browser and either click "Open trace file" or drag and drop your `trace.json` file directly onto the page.

Navigating the Perfetto UI

The Perfetto UI is organized around a central timeline view. The following table describes the key navigation controls:

Action	Keyboard Shortcut / Mouse Action
Zoom in	`W` key or scroll wheel up
Zoom out	`S` key or scroll wheel down
Pan left/right	Click and drag the timeline
Select an event	Single click on an event
View event details	Click an event; details appear in the bottom pane

Interpreting the Tracks

The Perfetto timeline is organized into tracks, where each track represents a different source of events.

Track	Description
`Processes`	Shows the different processes running during the profiling session
`GPU`	Shows GPU activity, with sub-tracks for each CUDA stream
`CPU`	Shows activity on each CPU core
`PyTorch Profiler`	Contains metadata, including `record_function` labels

Identifying a Bottleneck in Perfetto

The most common bottleneck visible in a Perfetto trace is **GPU idle time** — large gaps in the GPU track where no kernels are running. By zooming into these gaps and examining the CPU track, you can often see that the CPU is busy with data loading or preprocessing operations, starving the GPU of work.

---

5. Advanced Profiling APIs and Techniques

Beyond the basics, the PyTorch Profiler offers a suite of advanced features for more complex profiling scenarios.

Scheduling Profiling Runs

In many cases, you don't need to profile every single iteration of your training loop. The `torch.profiler.schedule` function allows you to specify exactly which iterations to profile, minimizing overhead while still capturing representative data.

```
my_schedule = torch.profiler.schedule(wait=1, warmup=1, active=3, repeat=2)

with torch.profiler.profile(schedule=my_schedule) as prof:
    for step in range(10):
        # ... your training code ...
        prof.step()  # Notify the profiler that a step has completed
```

The schedule parameters have the following meanings:

Parameter	Description
`wait`	Number of steps to skip at the start of each cycle
`warmup`	Number of steps to run the profiler but discard the results (to reduce initial overhead)
`active`	Number of steps to actively record
`repeat`	Number of times to repeat the wait/warmup/active cycle (0 = repeat indefinitely)

The `prof.step()` call is crucial for signaling the profiler to advance to the next state in its schedule.

Customizing Traces with `record_function`

The `record_function` context manager can be used to add granular, human-readable labels to any block of code. These labels appear in the profiler output and in the Perfetto trace, making it much easier to correlate the timeline with your source code.

```
with torch.profiler.profile(...) as prof:
    with torch.profiler.record_function("data_loading"):
        data = next(iter(data_loader))

    with torch.profiler.record_function("model_forward"):
        output = model(data)

    with torch.profiler.record_function("loss_and_backward"):
        loss = criterion(output, target)
        loss.backward()
```

Enabling Stack Trace Collection

Setting `with_stack=True` in the `profile` context manager instructs the profiler to record the Python call stack at the time of each operator call. This information is embedded in the trace and can be viewed in Perfetto, allowing you to trace a slow operator back to the exact line of code that triggered it.

```
with torch.profiler.profile(with_stack=True) as prof:
    # ... your code ...
```

Estimating Operator FLOPS

The profiler can estimate the number of floating-point operations (FLOPS) for certain operators, such as matrix multiplications and 2D convolutions. This is useful for understanding the computational complexity of your model.

```
with torch.profiler.profile(with_flops=True) as prof:
    # ... your code ...

print(prof.key_averages().table(sort_by="flops", row_limit=10))
```

---

6. Tutorial: From Profiling to Optimization — A Case Study

This tutorial provides a practical, end-to-end example of using the PyTorch Profiler to identify and fix a common performance bottleneck: a slow data loading pipeline.

The Problem: An Input-Bound Training Loop

Consider a training loop where the GPU is not being fully utilized because the data loading pipeline is too slow. We can simulate this by using a `DataLoader` with only a single worker process.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import torchvision.models as models
import torch.profiler

device = "cuda" if torch.cuda.is_available() else "cpu"
model = models.resnet18().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001)

# Simulate a slow data loader with only 1 worker
inputs = torch.randn(512, 3, 224, 224)
labels = torch.randint(0, 1000, (512,))
dataset = TensorDataset(inputs, labels)
slow_loader = DataLoader(dataset, batch_size=32, num_workers=1)
```

Profiling the Slow Loop

Now, let's profile this training loop to see the bottleneck in action.

```
with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
    ],
    schedule=torch.profiler.schedule(wait=1, warmup=1, active=3),
    on_trace_ready=torch.profiler.tensorboard_trace_handler('./log/slow'),
    record_shapes=True,
    with_stack=True
) as prof:
    for i, (data, target) in enumerate(slow_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        prof.step()
        if i >= 4:
            break
```

When you load the resulting trace into Perfetto, you will see large gaps in the GPU track. These gaps represent time when the GPU is idle, waiting for the next batch of data to be loaded from the CPU.

The Fix: Increasing DataLoader Workers

The solution is to increase the number of worker processes in the `DataLoader`. This allows data to be loaded and preprocessed in parallel with the GPU's computation, keeping the GPU fed with data.

```
# The fix: increase the number of workers
fast_loader = DataLoader(dataset, batch_size=32, num_workers=4)
```

After re-profiling with the `fast_loader`, the gaps in the GPU track should be significantly reduced, demonstrating the improvement in GPU utilization.

---

7. Tutorial: Profiling Distributed Training with HTA

Profiling distributed training jobs presents unique challenges. You need to understand not just the performance of a single GPU, but how all GPUs in your system are interacting. **Holistic Trace Analysis (HTA)** is an open-source tool developed for this purpose. [5]

Collecting Traces from Multiple Ranks

To profile a distributed job, you need to wrap the training loop on each rank with the PyTorch Profiler. It is crucial to save each rank's trace to a separate directory.

```
import torch.distributed as dist
import torch.profiler

def profiled_train(rank, world_size):
    dist.init_process_group("nccl", rank=rank, world_size=world_size)
    # ... model and data setup ...

    trace_dir = f"./log/rank_{rank}"
    with torch.profiler.profile(
        activities=[
            torch.profiler.ProfilerActivity.CPU,
            torch.profiler.ProfilerActivity.CUDA,
        ],
        schedule=torch.profiler.schedule(wait=1, warmup=1, active=3),
        on_trace_ready=torch.profiler.tensorboard_trace_handler(trace_dir),
    ) as prof:
        for i, (data, target) in enumerate(data_loader):
            # ... training step ...
            prof.step()
            if i >= 4:
                break

    dist.destroy_process_group()
```

Analyzing Traces with HTA

After collecting the traces, install HTA and use it to load and analyze all the rank traces together.

```
pip install HolisticTraceAnalysis
```

```
from hta.trace_analysis import TraceAnalysis

# Point HTA to the directory containing all rank trace subdirectories
analyzer = TraceAnalysis(trace_dir="./log/")
```

HTA provides several powerful analysis functions, summarized in the table below:

Function	Description
`get_temporal_breakdown()`	Breaks down GPU time into idle, compute, and non-compute categories
`get_idle_time_breakdown()`	Categorizes GPU idle time into host wait, kernel wait, and other
`get_gpu_kernel_breakdown()`	Analyzes the distribution of time across communication, compute, and memory kernels
`get_comm_comp_overlap()`	Measures the overlap between communication and computation, a key metric for distributed efficiency
`get_memory_bw_summary()`	Provides a summary of memory bandwidth utilization
`get_cuda_kernel_launch_stats()`	Identifies kernels with excessive launch overhead

A high communication-computation overlap is desirable, as it means the GPU is not sitting idle waiting for gradient synchronization to complete. If the overlap is low, it may indicate that your model's architecture or the distributed training configuration needs to be tuned.

---

8. Tutorial: Profiling Custom C++ and CUDA Operators

When you implement custom operators in C++ or CUDA, you can integrate them with the PyTorch Profiler using the `torch::autograd::profiler::RecordFunction` API. This allows your custom code to appear as named events in the profiler trace, giving you full visibility into its performance.

Instrumenting a C++ Operator

To add profiling support to a C++ operator, include the profiler header and create a `RecordFunction` object at the start of your function. The `RecordFunction` object uses RAII to automatically record the start and end of the function's execution.

```
#include <torch/extension.h>
#include <torch/csrc/autograd/profiler.h>

torch::Tensor my_custom_op(torch::Tensor input) {
    // This line creates a profiler event named "my_custom_op"
    torch::autograd::profiler::RecordFunction record("my_custom_op");

    // ... your operator implementation ...
    return input * 2;
}
```

Instrumenting a CUDA Kernel

The same approach applies to CUDA operators. You add the `RecordFunction` in the host-side C++ wrapper function, not inside the CUDA kernel itself.

```
#include <torch/extension.h>
#include <torch/csrc/autograd/profiler.h>

// Forward declaration of the CUDA kernel launcher
torch::Tensor my_cuda_kernel_launcher(torch::Tensor input);

torch::Tensor my_custom_cuda_op(torch::Tensor input) {
    torch::autograd::profiler::RecordFunction record("my_custom_cuda_op");
    return my_cuda_kernel_launcher(input);
}
```

Once your custom operator is built and registered, it will appear as a named event in the PyTorch Profiler trace whenever it is called, just like any built-in PyTorch operator.

---

9. Advanced Topic: Memory Profiling with Memory Snapshots

Understanding and optimizing memory usage is a critical aspect of deep learning. The modern approach to memory profiling in PyTorch revolves around **memory snapshots**, which provide a granular view of the memory allocator's state at a specific point in time.

> ***Note:*** *The* `export_memory_timeline` *function is deprecated and has known issues with eager mode models. The memory snapshot approach described in this section is the recommended replacement.*

Capturing a Memory Snapshot

A memory snapshot captures detailed information about every tensor currently live in memory, including its size, shape, and the Python callstack at the time of its allocation.

```python
import torch

# Enable memory history recording
torch.cuda.memory._record_memory_history(max_entries=100000)

# ... your model code ...

# Capture a snapshot
snapshot = torch.cuda.memory._snapshot()

# Save the snapshot to a file for analysis
import pickle
with open("memory_snapshot.pkl", "wb") as f:
    pickle.dump(snapshot, f)

# Stop recording
torch.cuda.memory._record_memory_history(enabled=None)
```

Analyzing Memory Snapshots

The captured snapshot can be visualized using the PyTorch memory visualization tool. You can upload the snapshot file to the online viewer at pytorch.org/memory_viz to get an interactive visualization of your model's memory usage over time.

This visualization shows memory allocations and deallocations as a timeline, categorized by the type of tensor (e.g., parameters, gradients, activations). This makes it easy to identify which parts of your model are using the most memory and where memory is being held longer than expected.

A Practical Memory Profiling Workflow

A systematic approach to memory profiling involves the following steps:

1. **Establish a baseline** by capturing a snapshot at the start and end of a training iteration to understand the steady-state memory usage.
2. **Identify memory growth** by comparing snapshots across multiple iterations. A steady increase in memory usage over time is a strong indicator of a memory leak.

3. **Pinpoint the source** by using the callstack information in the snapshot to identify the exact line of code responsible for the leak.
4. **Optimize and verify** by applying a fix and re-capturing snapshots to confirm the issue has been resolved.

---

10. Advanced Topic: Profiling `torch.compile`

`torch.compile` can significantly accelerate models through JIT compilation, but it introduces a new layer of abstraction that requires a specialized approach to profiling.

The `Torch-Compiled Region` Event

When you profile a model wrapped with `torch.compile`, you will see a `Torch-Compiled Region` event in your trace. This event represents the execution of the JIT-compiled graph. The individual PyTorch operators within this region are fused into optimized kernels and will not appear as separate events. [6]

Diagnosing Graph Breaks

A **graph break** occurs when `torch.compile` encounters a Python construct it cannot trace, forcing it to fall back to eager mode. Graph breaks are a common source of performance issues, as they prevent the compiler from optimizing across the break boundary.

You can identify graph breaks using `torch._dynamo.explain()`:

```
import torch._dynamo

explanation = torch._dynamo.explain(model)(inputs)
print(explanation)
```

The output will show you the number of graph breaks and the reason for each one, allowing you to refactor your code to eliminate them.

Combining Profiler and Compiler Diagnostics

The recommended workflow for profiling compiled models is:

1. **Profile the eager-mode model** to establish a performance baseline.
2. **Apply `torch.compile` and re-profile** to measure the overall speedup.
3. **Use `torch._dynamo.explain()`** to identify and eliminate graph breaks if the speedup is lower than expected.
4. **Use `TORCH_COMPILE_DEBUG=1`** environment variable to get detailed logs from the Inductor backend for low-level kernel analysis.

---

11. API Quick Reference

The following table provides a quick reference for the most important classes and functions in the `torch.profiler` module.

API	Description
`torch.profiler.profile(...)`	Main context manager for profiling sessions
`torch.profiler.ProfilerActivity.CPU`	Activity type for CPU events
`torch.profiler.ProfilerActivity.CUDA`	Activity type for CUDA GPU events
`torch.profiler.ProfilerActivity.XPU`	Activity type for Intel XPU events
`torch.profiler.schedule(wait, warmup, active, repeat)`	Returns a callable schedule for controlling profiling steps
`torch.profiler.tensorboard_trace_handler(dir)`	**Deprecated.** Saves traces for TensorBoard. Use Perfetto instead.
`torch.profiler.record_function(name)`	Context manager to add a custom label to a code block
`prof.key_averages()`	Returns aggregated profiler events, grouped by operator name
`prof.export_chrome_trace(path)`	Exports the trace to a Chrome/Perfetto-compatible JSON file
`prof.export_stacks(path)`	Exports stack traces to a file for flame graph visualization
`prof.step()`	Advances the profiler to the next step in its schedule
`torch.profiler.itt.mark(msg)`	Creates an instantaneous event in the trace (Intel ITT)
`torch.profiler.itt.range_push(msg)`	Pushes a named range onto the trace stack (Intel ITT)
`torch.profiler.itt.range_pop()`	Pops the current named range from the trace stack (Intel ITT)

12. References

1. pytorch/kineto: A CPU+GPU Profiling library
2. Profiler Integration — PyTorch main documentation
3. Perfetto UI — Open-source trace analysis tool
4. torch.profiler — PyTorch 2.10 documentation
5. Introduction to Holistic Trace Analysis — PyTorch Tutorials
6. Profiling to understand torch.compile performance — PyTorch documentation